CLOUD COMPUTING and DEVOPS (CSE 3130)
LAB MANUAL
5th Semester, B. TECH

School of Computer Engineering

SCHOOL OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that Ms./Mr. ……...……

Reg. No.: …..…… Section: …… Roll No.: ……

has satisfactorily completed the lab exercises prescribed for Cloud Computing & DevOps of Fifth Semester B.Tech, Degree at MIT, Manipal, in the academic year 2026-2027.

Date: .....

Signature of the Faculty Signature of Dean

Document Version Control

Sl. #	Date updated	Author/Editor	Summary of changes done/ Status
1	23-AUG-2026	Mohandas Shenoy P	Drafted Lab1 Finalized Lab1
2	31-AUG-2026	Soumya Nandan Mishra	Drafted Lab 2
3	01-AUG-2026	Mohandas Shenoy P	Drafted Lab 2 Finalized Lab 2
4	04-AUG-2026	Anup Bhat B	Drafted Lab 3
5	06-AUG-2026	Anup Bhat B Mohandas Shenoy P	Tested and updated the steps for Lab3
6	14-AUG-2026	Mohandas Shenoy P	Updated Lab4

COURSE OBJECTIVES

At the end of this course, the student should be able to:		No. of Contact Hours	Marks	Program Outcomes (POs)	Learning Outcomes (LOs)	BL *(Recommended)*
CLO1	To develop practical skills in version control strategies, automated continuous integration/continuous deployment (CI/CD) pipelines, and Infrastructure as Code (IaC) integration for DevOps workflows.	21	WIP	WIP	**WIP**	**WIP**
CLO2	To build proficiency in deploying, configuring, and managing containerized applications through container orchestration techniques to ensure system scalability and fault tolerance.	09	WIP	WIP	**WIP**	**WIP**
CLO3	To examine system performance monitoring, log analysis techniques, and the integration of cloud-native observability tools for effective operational incident response.	03	WIP	WIP	**WIP**	**WIP**
	Total	**33**	**WIP**	**WIP**	**WIP**	**WIP**

COURSE OUTCOMES (COs)

EVALUATION PLAN

Continuous Evaluation	60%	
1. Lab Examination	40%	

INSTRUCTIONS TO THE STUDENTS

Pre-Lab Session Instructions:

1. Students should carry the Lab Manual Book and the required stationery to every lab session
2. Be in time and follow the institution dress code
3. Must sign in the log register provided
4. Make sure to occupy the allotted seat and answer the attendance
5. Adhere to the rules and maintain the decorum

In-Lab Session Instructions:

- Follow the instructions on the allotted exercises
- Show the program and results to the instructors on completion of experiments
- On receiving approval from the instructor, copy the program and results in the lab record
- Prescribed textbooks and class notes can be kept ready for reference if required

General Instructions for the exercises in Lab:

- Implement the given exercise individually and not in a group.
- The programs should meet the following criteria:
  - Programs should be interactive with appropriate prompt messages, error messages if any, and descriptive messages for outputs.
  - Programs should perform input validation (Data type, range error, etc.) and give appropriate error messages and suggest corrective actions.
  - Comments should be used to give the statement of the problem and every function should indicate the purpose of the function, inputs and outputs.
  - Statements within the program should be properly indented.
  - Use meaningful names for variables and functions.
  - Make use of constants and type definitions wherever needed.
- Plagiarism (copying from others) is strictly prohibited and would invite severe penalty in evaluation.

- In case a student misses a lab class, he/she must ensure that the experiment is completed during the repetition class in case of genuine reason (medical certificate approved by HOD) with the permission of the faculty concerned
- Questions for lab tests and examination are not necessarily limited to the questions in the manual, but may involve some variations and/or combinations of the questions.
- A sample note preparation is given as a model for observation.

Students Should Not

Bring mobile phones or any other electronic gadgets to the lab.
Go out of the lab without permission.

LAB EXPERIMENTS

LAB #1

LAB #1: Configure Nginx Webserver on local Linux Machine to serve static and web pages via PHP and JSP

Objectives:

1. To be familiar with Linux commands
2. To learn about web servers Nginx , PHP-FPM , Java Container Tomcat
3. To understand how to configure a web server to serve static and web pages via different programming languages
4. To learn to check the server logs

Conceptual Background:

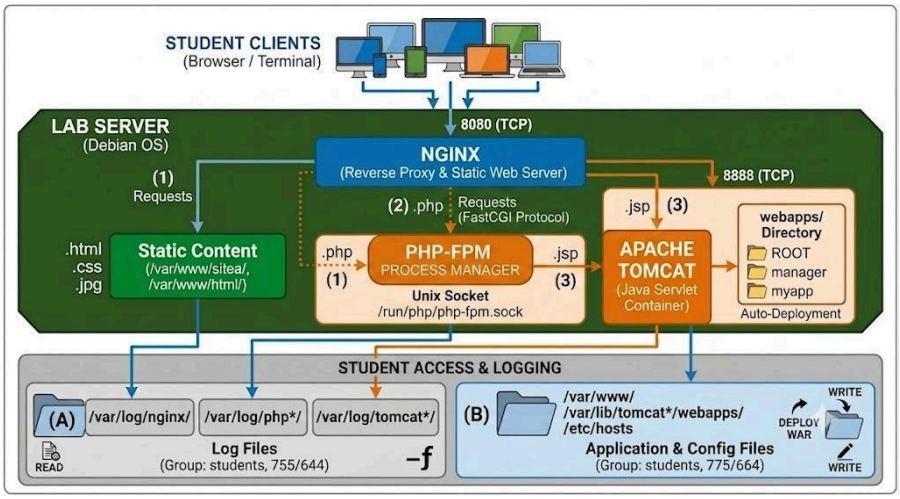


Figure 1: Lab1 setup

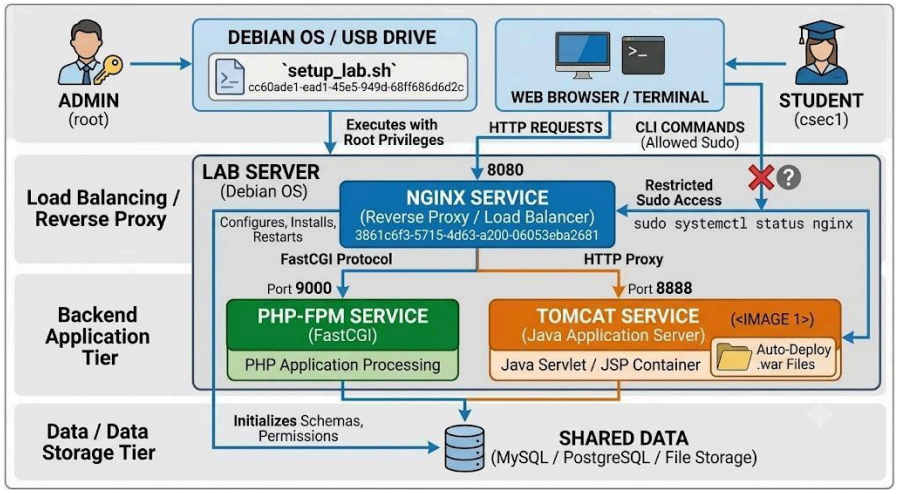


Figure 2: Typical 3-tier architecture

Nginx: Event-Driven Reverse Proxy & Web Server

Unlike traditional web servers (like Apache httpd) that spawn a dedicated thread or process for every incoming connection (a thread-per-connection model), Nginx utilizes an asynchronous, event-driven, non-blocking architecture.

Key Architectural Concepts:

- Master-Worker Architecture: Nginx runs one Master Process (which reads configurations and maintains worker processes) and multiple Worker Processes (which handle actual network connections).
- Event Loop: A single worker process uses OS-level I/O multiplexing mechanisms (such as epoll in Linux) to handle thousands of concurrent HTTP connections on a single thread with minimal RAM and CPU overhead.
- Primary Roles:
    - Static File Server: Serves HTML, CSS, JavaScript, and images directly from disk with extreme speed.
    - Reverse Proxy: Accepts client requests on port 8080 (or 80) and forwards them to backend application servers (like PHP-FPM or Tomcat).
    - Load Balancer: Distributes incoming traffic across multiple backend application instances.

PHP Application Processing: FastCGI & PHP-FPM

Nginx cannot process dynamic code (like .php scripts) natively. It only acts as a delivery mechanism. To execute PHP code, Nginx relies on an external process manager using the FastCGI protocol.

Key Architectural Concepts

1. CGI vs. FastCGI:

    - CGI (Common Gateway Interface): Spawns a new OS process for every single HTTP request, leading to massive memory overhead.
    - FastCGI: Keeps long-running worker processes alive in memory to execute incoming scripts sequentially, avoiding process creation overhead.
2. PHP-FPM (FastCGI Process Manager):
    - PHP-FPM is an advanced implementation of FastCGI for PHP.
    - It maintains a dynamic pool of worker processes waiting to accept requests.
3. How Nginx Interacts with PHP-FPM:
    - Client requests http://localhost:8080/index.php.
    - Nginx checks the location block matching .php.
    - Nginx translates the HTTP request into FastCGI binary format and sends it over a Unix Domain Socket or TCP/IP Socket (typically 127.0.0.1:9000) to PHP-FPM.
    - PHP-FPM executes the script, captures output, and returns HTML back to Nginx, which sends it to the browser.

Apache Tomcat: Java Servlet & JSP Container

Apache Tomcat is an open-source Java Servlet Container (and Web Server) that implements Jakarta Servlet, Jakarta Expression Language, and WebSocket technologies. Unlike Nginx or standard Apache HTTP, Tomcat is specifically designed to execute compiled Java code packaged inside .war (Web Application Archive) files.

Key Architectural Concepts

- Servlet Container (Catalina): Catalina is the core engine of Tomcat that handles request lifecycle management, routing incoming requests to the appropriate Java Servlet class, instantiating Java objects, and managing sessions.
- JSP Engine (Jasper): Converts JavaServer Pages (.jsp) into standard Java Servlets on-the-fly, compiles them into bytecodes (.class), and executes them.
- Deployment (.war Files): A .war file is a compressed ZIP archive containing compiled Java bytecode, web configuration descriptors (web.xml), and static assets. Tomcat automatically uncompresses and deploys any .war file placed into its webapps/ folder.
- Architecture Role: Tomcat usually listens on a dedicated internal port (like 8888) and handles business logic/database operations. Nginx sits in front of Tomcat to handle TLS termination, static asset caching, and security filtering.

Solved Examples:

General Instructions:

- Perform all steps logged into your assigned student user account (e.g., csec1). Do not log in as root.
- When managing services or network interfaces, prefix commands with sudo. You will not be prompted for a password for authorized lab commands.

- You may use nano or vi in the terminal or launch VSCode to edit configuration files.
- Nginx (8080) and Tomcat (8888) is already installed on your machine and running at specific ports mentioned.
- You can download sample.war from Lab Shared server directory '**1_Lab_files'**

1) Configure Nginx webserver to serve static web page (Name-Based Virtual Hosting)

Name-based virtual hosting allows a single Nginx web server instance to host multiple domain names (e.g., sitea.local) on a single IP address.

Step 1.1: Map Domain Names Locally

Since local development domain names are not registered on public DNS servers, map sitea.local to your local loopback address (127.0.0.1).

- Open /etc/hosts with administrative rights:
  `nano /etc/hosts`

- Add the following line at the end of the file:
  `127.0.0.1 sitea.local`

Step 1.2: Create the Web Document Root

- Create a dedicated folder for sitea.local and add an HTML entry point.

  Create the web root directory:

  `mkdir -p /var/www/sitea`

- Create an index.html file inside the new folder:

  `nano /var/www/sitea/index.html`

  Paste the following HTML sample:

```
<!DOCTYPE html>
<html>
<head>
      <title>Welcome to Site A</title>
</head>
<body>
       <h1>Success! Name-Based Virtual Host (Site A) is
working!</h1>
</body>
</html>
```

Step 1.3: Create the Nginx Server Block Configuration

- Define how Nginx routes web traffic for sitea.local.

  Create a new server block file in sites-available:

  ```
  nano /etc/nginx/sites-available/sitea.conf
  ```

  Add the following configuration block:

  ```
  server {
      listen 8080;
      server_name sitea.local;

      root /var/www/sitea;
      index index.html;

      location / {
          try_files $uri $uri/ =404;
      }
  }
  ```

Step 1.4: Enable the Configuration and Reload Nginx

- Link the configuration file to the active sites-enabled directory and apply changes.

  Create a symbolic link to activate the site:

  ```
  ln -s /etc/nginx/sites-available/sitea.conf
/etc/nginx/sites-enabled/
  ```

- Verify that your configuration syntax contains no errors:

  ```
  sudo nginx -t
  ```

- Reload Nginx to apply the new Virtual Host:

  ```
  sudo systemctl reload nginx
  ```

Step 1.5: Verification

- Open web browser and navigate to:
  `http://sitea.local:8080`

- Alternatively, test the HTTP response via terminal:
  `curl http://sitea.local:8080`
  (Expected output: The HTML content created in Step 1.2).

Step 1.6: View error and access logs

- view errors

```
tail -f /var/log/nginx/error.log
```

- view incoming requests

```
tail -f /var/log/nginx/access.log
```

2) Configure Nginx webserver to serve Dynamic Pages via PHP(FastCGI/PHP-FPM)
Nginx processes dynamic scripts by passing requests to an external application processing pool (PHP-FPM) via the FastCGI protocol.

Step 2.1: Create the PHP Web Document Root

- Create a directory for your PHP web files:

```
mkdir -p /var/www/php_site
```

- Create an index.php test file:
  `nano /var/www/php_site/index.php`

- Add code to display detailed PHP runtime environment information:

```
<?php
      echo "<h1>Nginx + PHP-FPM FastCGI Execution Successful</h1>";
      phpinfo();
?>
```

Step 2.2: Identify the Installed PHP-FPM Socket

- Nginx communicates with PHP-FPM through a Unix domain socket. Identify the active PHP version socket on your machine:

  ```
  ls /run/php/php*-fpm.sock
  ```

  *(Example output path: /run/php/**php8.2-fpm.sock**). Note your exact version number.*

Step 2.3: Configure Nginx for FastCGI Processing

- Create a new configuration file for the dynamic site:
  `nano /etc/nginx/sites-available/php_site.conf`

- Add the configuration block, updating the socket path to match Step 2.2:

```
server {
    listen 8080;
    server_name phpsite.local;

    root /var/www/php_site;
    index index.php index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    # Pass PHP scripts to FastCGI server via PHP-FPM Unix socket
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
    }
}
```

- Map phpsite.local in /etc/hosts:

```
nano /etc/hosts

Add following at the end of the file

127.0.0.1 phpsite.local
```

Step 2.4: Enable Site and Test Configuration

- Enable the PHP site:

```
ln -s /etc/nginx/sites-available/php_site.conf
/etc/nginx/sites-enabled/
```

- Check configuration syntax:

```
sudo nginx -t
```

- Reload Nginx:

```
sudo systemctl reload nginx
```

Step 2.5: Verification

- Open your browser and navigate to: http://phpsite.local:8080

Confirm that the page renders the dynamic PHP Version Information dashboard.

- Alternatively, test the HTTP response via terminal:
  curl http://phpsite.local:8080
  (Expected output: It should render the contents mentioned in Step 2.1 with php server configuration list).

Step 2.6: View error and access logs

- view errors

```
tail -f /var/log/nginx/error.log
```

- view incoming requests

```
tail -f /var/log/nginx/access.log
```

Exercises:

Record your findings and understandings in the observation book

1) Configure Nginx webserver to server static web page(IP-Based Virtual Hosting)

Notes:

- You can use secondary IP Address: assign 127.0.0.2 (or 127.0.0.3) as your secondary IP.
- You have sudo permissions for ip addr add, you can run following
  sudo ip addr add 127.0.0.2/8 dev lo

2) Configure Nginx webserver to server dynamic Pages via JSP(OpenJDK + Tomcat Reverse Proxy)

Notes:

- Tomcat listens on port 8888. Configure Nginx to proxy requests on port 8080 to Tomcat's port 8888
- If you deploy a sample.war file or a raw JSP folder directly, it goes into /var/lib/tomcat10/webapps/
- If JSP app fails, you can check logs at /var/log/tomcat10/catalina.out.

Remarks:

After completion of Exercises, run the following (mandatory)

```
sudo lab-reset
```

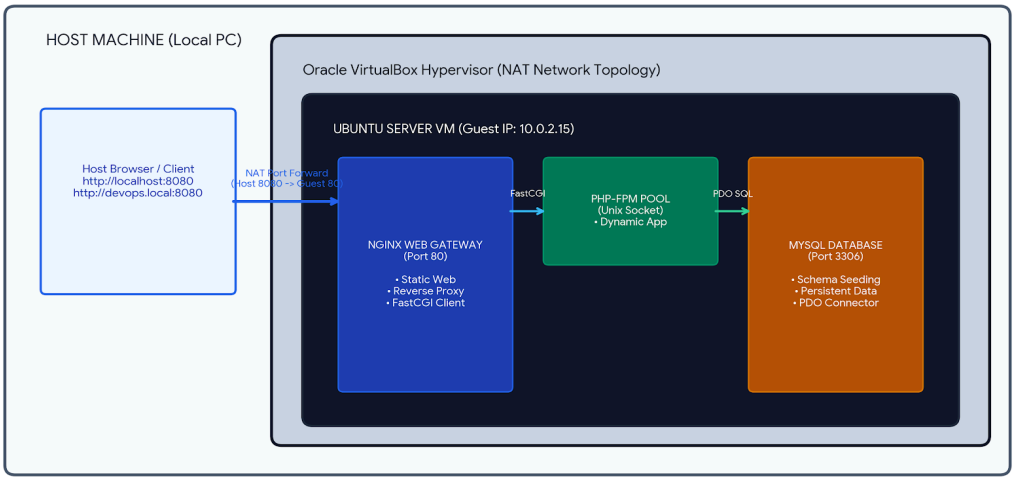
LAB #2

LAB #2: Provisioning an isolated virtualization environment using Oracle Virtual Box & deploy 3-tier LEMP stack solution

Objectives:

1. To gain hands-on experience with hypervisor-based virtualization using Oracle Virtual Box.
2. To configure guest-to-host virtual network topologies using NAT and Port Forwarding.
3. To deploy and manage a multi-tier LEMP stack (Linux, Nginx, MySQL, PHP-FPM) inside an isolated guest environment.
4. To examine Layer 7 name-based ingress routing, process failure modes, and system resource monitoring under load.

Conceptual Background:



1. ### Hardware Virtualization with Type-2 Hypervisor:

   While Lab 01 configured web daemons directly on local host operating systems, modern cloud infrastructure relies on hypervisors. A Type-2 Hypervisor (such as Oracle Virtual Box) runs on top of a host operating system and virtualizes hardware resources (CPUs, Memory, Storage, and Virtual Network Adapters) to run fully isolated guest operating systems (For example Ubuntu Desktop or Server).

2. ### Network Virtualization (NAT and Port Forwarding):

   By default, Virtual Box isolates virtual machines behind a Network Address Translation (NAT) virtual router. The virtual machine receives an isolated internal IP address (e.g., 10.0.2.15) that cannot be accessed directly from the host network or external client browsers. To enable access to guest web applications, NAT Port Forwarding establishes a Layer 4 socket mapping rule at the hypervisor level. Requests sent to a designated port on the physical host (e.g., 127.0.0.1:8081) are intercepted by the Virtual Box process and translated to port 80 inside the virtual machine instance.

3. 3-Tier LEMP Stack Architecture:
   A production LEMP stack partitions dynamic web hosting into 3 layers:
   - Presentation/Ingress Layer (Nginx): An asynchronous, event-driven web gateway listening on Port 80. It serves static assets directly and routes dynamic execution requests via the FastCGI protocol.
   - Application Layer (PHP-FPM): The FastCGI Process Manager runs an isolated worker pool communicating with Nginx over a local Unix domain socket (/var/run/php/php8.3-fpm.sock), avoiding process-spawning overhead.
   - Data Persistence Layer (MySQL Server): A relational database engine listening on Port 3306 that manages relational schemas, user privileges, and persistent records via dynamic PHP PDO drivers.

4. Cloud Ingress and Name-based Virtual hosting:
   In modern cloud environments (such as Kubernetes Ingress Controllers or AWS Application Load Balancers), a single virtual gateway instance routes incoming traffic to multiple backend microservices based on the **HTTP Host: header**. Nginx matches incoming host request strings (e.g., devops.local vs. analytics.local) to distinct server { ... } configuration blocks, providing multi-tenant application routing on a single public port.

Solved Examples:

Step 1: Virtual Machine Provisioning in Oracle Virtual Box

- Download Ubuntu Server VDI(pre-installed Ubuntu Server OS) from https://sourceforge.net/projects/osboxes/files/v/vb/59-U-u-svr/25.04/64bit.7z/download
  OR
  Download **64bit.7z** from Lab Shared server directory '**2_Lab_files'** and extract it
- Launch Oracle Virtual Box and click New.
  Note: Remove any existing Virtual Machines (right click and select remove)
- Virtual machine name and operating system
  - VM Name: Lab2_VM_LEMP
  - VM Folder: Choose the directory where .VDI is saved
  - ISO Image: Leave empty/None since we are using pre-installed VDI file
  - OS: Linux
  - OS Distribution: Ubuntu
  - OS Version: Ubuntu (64-bit)
- Specify virtual hardware
  - Base Memory: 8000 MB RAM (8 GB)
  - Number of CPUs: 12 CPUs
- Specify virtual hard disk
  - Select the radio button "Use an Existing Virtual Hard Disk File".
  - Click the folder icon with the green arrow next to the dropdown menu to open the Hard Disk Selector.
  - Click Add at the top left, browse to your extracted .vdi file, select it, and click Open.
  - Highlight the .vdi file in the selector list and click Choose.

- Click Next, review the summary, and click Finish

Step 2: Hypervisor NAT Network & Port Forwarding Setup

- Navigate to VM Settings > Network > Adapter 1 > Click “Port Forwarding”
- Add Port forwarding rule:
  - Name: HTTP
  - Protocol: TCP
  - Host Port: 8081
  - Guest Port: 80

  Note: Host OS means the base Operating System (Debian), Guest OS means Ubuntu OS running inside VM

Step 3: Boot and Log In

- Select your new VM from the left list and click Start.
  Note: if you encounter any error while starting the VM, close Oracle Virtual Box, then execute following in Host OS terminal
  - *sudo apt update && sudo apt install -y linux-headers-amd64 linux-headers-$(uname -r) build-essential dkms*
  - *sudo /sbin/vboxconfig*
  - *echo "blacklist kvm_intel" | sudo tee -a /etc/modprobe.d/blacklist.conf*
  - *echo "blacklist kvm" | sudo tee -a /etc/modprobe.d/blacklist.conf*
- When the system boots up to the terminal login prompt, enter the default OSBoxes credentials:
  - Username: *osboxes.org*
  - Password: *osboxes.org*
- (Optional step) Launch Firefox Browser for Your Lab(Optional if its not installed inside VM
  Execute following from terminal inside VM
  - *sudo apt update && sudo apt install -y xorg firefox*
  - *startx firefox*
- (Optional step) Enable copy and paste between your Debian host OS and Ubuntu Server VM in VirtualBox
  - Configure VirtualBox settings and install Guest Additions. Google for this !:)
    OR
  - Server administration is SSH
    - In VirtualBox VM Settings → Network → Advanced → Port Forwarding: Add a rule: Host Port 2222 → Guest Port 22.
    - Install OpenSSH server inside the VM: *sudo apt install -y openssh-server*
    - Open a terminal on your Debian host and run:
      - *ssh osboxes@localhost -p 2222*

- Now you can copy and paste following commands directly in your host's native terminal OR execute from Terminal from Inside Guest OS
    - Update repository indexes and install Nginx, PHP-FPM, and MySQL server:
      sudo apt update && sudo apt install nginx php-fpm php-mysql mysql-server -y
    - Verify daemon execution status:
      sudo systemctl status nginx
      sudo systemctl status php-fpm*
      sudo systemctl status mysql

Step 4: Nginx FastCGI Server Block Configuration

- Edit the default server block file:
  sudo nano /etc/nginx/sites-available/default
- Replace content with the FastCGI processing block:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    root /var/www/html;
    index index.php index.html index.htm;
    server_name _;
    location / {
        try_files $uri $uri/ =404;
    }
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/var/run/php/php8.3-fpm.sock; # Adjust socket version if required
    }
    location ~ /\.ht {
        deny all;
    }
}
```

- Validate syntax and reload Nginx:
  sudo nginx -t && sudo systemctl reload nginx

Exercises:

Exercise #1: Find Guest IP , Guest Port and Remote IP details

- Create /var/www/html/server_info.php inside the VM:

```
<?php
        echo "<h3>Server Infrastructure Details</h3>";
        echo "VM Guest IP Address: " . $_SERVER['SERVER_ADDR'] . "<br>";
        echo "Client Remote Access IP: " . $_SERVER['REMOTE_ADDR'] . "<br>";
        echo "Gateway Server Port: " . $_SERVER['SERVER_PORT'];
?>
```

- Access server_info.php from Host OS and record your findings.
    - VM Guest IP Address: _____
    - Client Remote Access IP: _____
    - Gateway Server Port: _____
- Access server_info.php from Guest OS and record your findings.

- VM Guest IP Address: _____
- Client Remote Access IP: _____
- Gateway Server Port: _____

Exercise #2: Analyze Server/Process failure

- Stop PHP-FPM daemon (sudo systemctl stop **php8.4-fpm**) and refresh web page http://localhost:8081/server_info.php in host browser. Record following findings
  - Error displayed & Error thrown in error.log
  - Write down your observations
- Restart PHP-FPM (sudo systemctl start **php8.4-fpm**) and stop Nginx (sudo systemctl stop nginx) and refresh web page http://localhost:8081/server_info.php in host browser. Record following findings
  - Error displayed
  - Write down your observations

  Note: check php-fpm version and update above php-fpm start/stop command accordingly

Exercise #3: System Resource Utilization & Workload Monitoring

- Open a second terminal inside the VM and launch process monitor: htop
  - Record following findings
    - How many # of CPUs you can see in the screen ? _____
    - Baseline CPU Usage : ___ %
    - Baseline Memory Usage: ___ GB/__GB

  Note: if htop is not working , you can install it by typing *sudo apt install htop*
- Execute 5000 requests from host machine command line:

*for i in {**1..5000**}; do curl -s http://localhost:8081/server_info.php > /dev/null; done*

  - Record following findings
    - Peak Workload CPU Usage: ___%
    - Peak Workload Memory Usage: ___ GB/__GB
- Which process from following registered higher CPU utilization spike?
  - 'nginx: worker process'
  - 'php-fpm: pool www'

Additional Exercises(optional):

Exercise #4: 3-Tier Database Connectivity & Data Persistence

- Access MySQL CLI inside VM (*sudo mysql*) and seed schema:

  -- 1. Create the lab database
  CREATE DATABASE devops_lab;

  -- 2. Create a dedicated database user with a secure password
  CREATE USER 'lab_user'@'localhost' IDENTIFIED BY 'LabPassword123!';

-- 3. Grant full permissions on devops_lab database to lab_user

```
GRANT ALL PRIVILEGES ON devops_lab.* TO 'lab_user'@'localhost';
FLUSH PRIVILEGES;
```

-- 4. Switch to the newly created database

```
USE devops_lab;
```

-- 5. Create the table schema

```
CREATE TABLE student_records (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    roll_no VARCHAR(20) NOT NULL
);
```

-- 6. "SEED" the schema (Insert sample initial records)

```
INSERT INTO student_records (student_name, roll_no) VALUES ('Student 1', '1001');
INSERT INTO student_records (student_name, roll_no) VALUES ('Student 2', '1002');
INSERT INTO student_records (student_name, roll_no) VALUES ('Student 3', '1003');
INSERT INTO student_records (student_name, roll_no) VALUES ('Student 4', '1004');
INSERT INTO student_records (student_name, roll_no) VALUES ('Student 5', '1005');
EXIT;
```

- Create /var/www/html/db_test.php using PDO connection logic

```
<?php
        $dsn = "mysql:host=127.0.0.1;dbname=devops_lab;charset=utf8mb4";
        try {
            $pdo = new PDO($dsn, 'lab_user', 'LabPassword123!');
            echo "<h3 style='color:green;'>MySQL Database Connection Successful!</h3>";
            $stmt = $pdo->query('SELECT id, student_name, roll_no FROM student_records');
            echo "<table border='1'><tr><th>ID</th><th>Name</th><th>Roll No</th></tr>";
            while ($row = $stmt->fetch()) {
                echo
        "<tr><td>{$row['id']}</td><td>{$row['student_name']}</td><td>{$row['roll_no']}</td></tr>";
            }
            echo "</table>";
        } catch (PDOException $e) {
            echo "<h3 style='color:red;'>Database Connection Failed: " . $e->getMessage() . "</h3>";
        }
?>
```

- Verify at http://localhost:8081/db_test.php
- Record following findings
    - Page response
    - Stop MySQL service (*sudo systemctl stop mysql*) and record the Page response

Exercise #5: Cloud Ingress Routing — Multi-Site Name-Based Virtual Hosting

- Create web roots /var/www/devops and /var/www/analytics with corresponding index files.
- Configure site blocks for devops.local and analytics.local under /etc/nginx/sites-available/
- Create Symbolic links both site files to /etc/nginx/sites-enabled/ , remove default site, and reload Nginx.
- Add loopback entries into Host Machine hosts file.

    - 127.0.0.1 devops.local
    - 127.0.0.1 analytics.local
- Verify accessing the files from host OS browser
    - http://devops.local:8082/devops.html
    - http://analytics.local:8083/analytics.html
- Record following findings
    - Specific HTTP Request Header used by Nginx to differentiate virtual site hosts

Remarks

- Power off the Virtual Machine: Inside the Ubuntu VM terminal, run:
  sudo poweroff
- Delete the VM Instance and Disk Files:
    - Open Oracle VM VirtualBox Manager on your host machine.
    - Right-click on the **Lab2_VM_LEMP** virtual machine in the left panel.
    - Select Remove
    - Click Delete all files (this permanently removes the virtual hard disk .vdi file and associated VM settings from the lab machine).

LAB #3

LAB #3: Implementing RESTful Web Service using Spring Boot and Swagger (OpenAPI)

Objectives:

1. To develop a RESTful Web Service using Java and Spring Boot.
2. To automatically generate the OpenAPI specification for the developed RESTful Web Service.
3. To document and test the RESTful Web Service using Swagger UI.

Conceptual Background:

Representational State Transfer (REST) is an architectural style for developing distributed applications that communicate over the HTTP protocol. In a RESTful Web Service, functionality is exposed through resources that are identified using Uniform Resource Identifiers (URIs) and are accessed using standard HTTP methods such as GET, POST, PUT, and DELETE. Unlike SOAP-based Web Services, REST does not require XML-based messaging or a WSDL contract, making it lightweight, platform independent, and easy to integrate with applications developed using different programming languages.

Solved Examples:

In this experiment, a RESTful Web Service is developed using the Spring Boot framework. The project is first created as a Maven application, which automatically manages all the external libraries required for building and publishing the service. The application is then configured using Spring Boot dependencies that provide an embedded Tomcat web server and the Spring MVC framework for handling HTTP requests. A REST controller is subsequently implemented to expose a simple calculator operation as a REST endpoint.

Once the application is executed, Spring Boot automatically starts the embedded Tomcat server and scans the project for REST controller classes. It identifies methods annotated with request-mapping annotations such as @GetMapping and registers the corresponding URL mappings. Consequently, when a client sends an HTTP request to a particular endpoint, the request is routed to the appropriate controller method, the required computation is performed, and the result is returned as an HTTP response.

To facilitate API documentation, the project is also configured with the OpenAPI library. During application startup, the library scans the REST endpoints and automatically generates an OpenAPI specification describing the available operations, their parameters, and response types. This

specification is made available as a JSON document and is subsequently used by Swagger UI to generate interactive API documentation. The service is finally tested both by directly invoking the endpoint through a web browser and by using Swagger UI, which allows the available REST operations to be executed without developing a separate client application.

Step 1: Download project source code

- 'calculator-spring.zip' from Lab Shared Server folder '**3_Lab_files'**
- Unzip and open this folder from VSCode
- Ensure project structure as follows

```
src/main/java
└── com/example/calculator
    ├── CalculatorApplication.java
    └── CalculatorController.java
```

Step 2: Install maven, Postman

Open Terminal, execute commands (download **lab3commands.txt** from folder '**3_Lab_files'**)

Step 3: Update CalculatorController add operations

Implement /add using @RestController, @GetMapping, @RequestParam and @Operation.

Step 4: Compile and Run

mvn clean compile
mvn spring-boot:run

Step 5: Open Browser and verify REST Service

Open http://localhost:8080/add?a=10&b=20 and verify output = 30

Step 6: Inside Postman add following link view and test the API

http://localhost:8080/v3/api-docs

Step 7: Open Browser and test REST service with Swagger UI

Open http://localhost:8080/swagger-ui/index.html and execute the Add operation.

Exercises:

1. Modify only the implementation of add() and verify whether the OpenAPI document changes.
2. Add subtract(), multiply() and divide() operations. Use POST , UPDATE methods
3. Change the endpoint path and observe the OpenAPI changes. (E.g.: Let all the service operations begin from /calculator) and test all operations using Swagger
4. Use Postman to invoke the RESTful Web Service and observe the HTTP request, response body, response status code, and headers and test all operations
5. Explain @RestController, @GetMapping, @RequestParam and @Operation.
6. Summarize the role of Spring Boot and Maven.
7. Illustrate the lifecycle of a REST request.

Remarks

- Delete the project once all exercises are complete

LAB #4

LAB #4: Version Control, Branching Strategies, and Collaboration using Git and GitHub

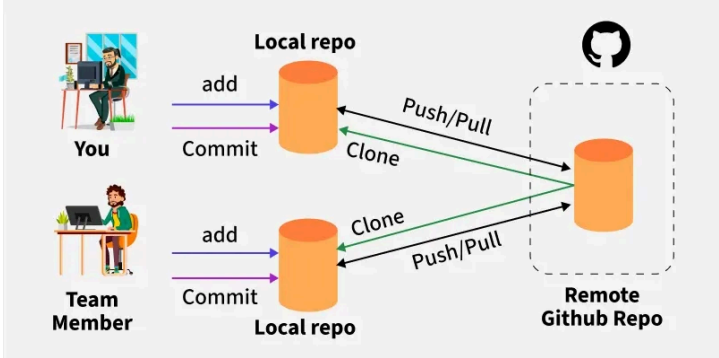
4.1 Objectives:

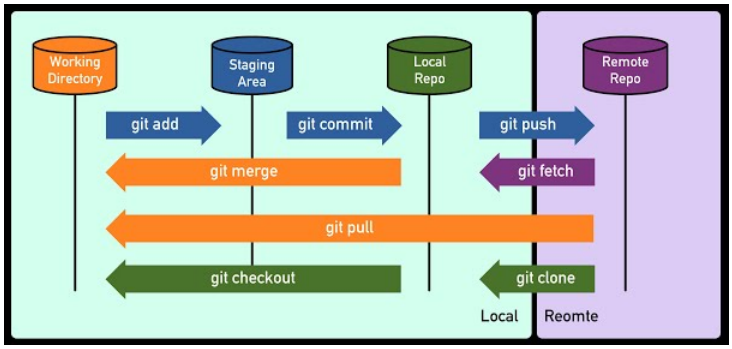
1. To initialize, configure, and manage local Git repositories using essential commands such as git init, git config, git add, git commit, and git status.
2. To inspect commit history with git log and manage file states using git restore and reset mechanisms (-soft, --mixed, --hard).
3. To implement isolated feature development and code integration using Git branching, switching, merging, and git rebase strategies.
4. To resolve merge conflicts manually and manage temporary uncommitted modifications using git stash operations (stash, pop, drop, list).
5. To configure GitHub authentication using Personal Access Tokens (PAT) and perform remote collaboration workflows, including repository creation, remote linking, pushing (git push), and cloning (git clone).

4.2 Theory/Concepts:

Git is a distributed version control system designed to track changes in source code locally across development lifecycles. It enables developers to record project revisions, create isolated feature branches, and inspect history without requiring an active network connection.

GitHub, on the other hand, is a cloud-based hosting platform that provides remote repository management and collaborative tools built on top of Git. It extends Git's core capabilities by offering pull requests, code reviews, issue tracking, and access management. While Git manages version control mechanics directly on your local workstation, GitHub serves as the centralized hub for team synchronization and remote collaboration. Together, they form the foundational backbone of modern collaborative software development and DevOps pipelines.





4.2.1: The Three States and Areas of Git

Git tracks project files across three primary stages:

Working Directory: The local filesystem where files are created and edited.

Staging Area (Index): An intermediate staging environment that formats and prepares changes to be included in the next commit via git add.

Commit History (Repository): The permanent chronological record of cryptographically hashed snapshots saved via git commit.

4.2.2: Undoing Changes and Reset Modes

Git provides mechanisms to unstage files, discard modifications, or alter repository state relative to the HEAD pointer (the reference to the latest commit of the current active branch):

Soft Reset (git reset --soft HEAD~1): Rewinds the commit history by one commit while preserving all changes directly inside the Staging Area.

Mixed Reset (git reset --mixed HEAD~1): The default reset mode that undoes the commit and unstages changes, keeping modifications strictly in the Working Directory.

Hard Reset (git reset --hard HEAD~1): Completely removes the last commit, clears the Staging Area, and permanently deletes uncommitted working changes.

4.2.3: Branching, Merging, and Stashing

Branching & Merging: Feature branching (git checkout -b <branch>) enables isolated code development without impacting the main timeline. Once complete, git merge integrates feature changes into the base branch.

Merge Conflicts: Occur when concurrent branches modify the same lines of a file; Git requires manual intervention via an editor to reconcile conflicting blocks before completing the merge.

Stashing (git stash): Temporarily shelves uncommitted working directory and staging modifications to a hidden stack, enabling clean branch switching without losing in-progress work.

4.2.4: Git Rebasing

Unlike git merge which combines histories using an explicit merge commit, git rebase rewires and moves the base of a feature branch to the tip of another branch. This replays feature commits sequentially on top of the updated target branch to maintain a clean, linear commit history.

4.2.5: Remote Repositories and GitHub Integration

Remote Synchronization: Local repositories link to central platforms like GitHub using remote origin aliases (git remote add origin <url>), enabling distributed pushing (git push -u origin

<branch>) and cloning (git clone <url>).
Authentication: Access to remote repositories is secured via HTTPS using fine-grained Personal Access Tokens (PAT) configured with read and write permissions to repository contents.

4.3 Solved Examples:

None

4.4 Exercises:

General Instructions:

- Register in github.com
- Choose Dabian OS and Login as MTECH user
- Tools used VSCode, Git, GitHub

4.4.1: Create a Project and make Git to track changes in this project

- Create a directory my-website-*studentregno*
- Open this directory in VSCode
- Create index.html, add following contents

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Website</title>
</head>
<body>
<h1>welcome to my website</h1>
</body>
</html>
```

- In Terminal window, type to check the status: *git status*
  What did you observe in the output? _____
- Type to get this project tracked by git: *git init*
  What did you observe in the output? _____
- Type to list files and directories: ls -a
  What did you observe in the output? _____
- Type to check the status: *git status*
  What did you observe in the output? _____
- Type to add the untracked files into staged area: *git add .*
  What does this command do? _____
- Type to check the status: *git status*
  What did you observe in the output? _____
- Identify yourself who you are [One-time setup for every project]
  Type to set your email: *git config user.email "student.email@example.com"*
  Type to set your name: *git config user.name "Student Name"*
  Type to list the settings: *git config --list*
  Note: Change it your email and name accordingly
- Type to commit the changes: *git commit -m "added index.html"*
- Type to check the status: *git status*

What did you observe in the output? _____

4.4.2: Making new changes and checking the history of commits

- Modify index.html, add following line
  <link rel="stylesheet" href="style.css">
  - Type if you want to discard these changes: *git restore index.html*
- Type to add the untracked files into staged area: *git add .*
  - Type if you want to unstage these changes: *git restore --staged index.html*
- Add new file to the project *style.css*

```
h1 {
   background-color: red;
}
```

- Type to add the untracked files into staged area: *git add index.html style.css*
- Type to check the status: *git status*
  What did you observe in the output? _____
- Now proceed with committing these changes with proper comments
- Type to display list of commits done: *git log –oneline*
  What did you observe in the output? _____

4.4.3: Reversing the changes

It lets you to unstage the files, discard local edits, or move project back to a previous commit. It is a way to control the 3/three state of Git: Working Directory (local files), Staging Area, and Commit History

- Soft Reset (*git reset --soft HEAD~1*)
  - Moves repository back by one commit but keeps all your changes.
  - Modified files are safely placed back into the Staging Area (ready to commit again).
  - Use this, if you had committed too early, forgot a file, or want to rewrite your last commit message.
- Mixed Reset (*git reset --mixed HEAD~1*)
  - This is the default option if you don't type a flag. It undoes the commit and unstages your changes.
  - Your changes are kept safely in your Working Directory, but they are not staged.
  - Use this, if you want to completely redo your files and stage them one by one.
- Hard Reset (*git reset --hard HEAD~1*)
  - Erases the last commit, unstages your changes, and deletes your changes.
  - Your files match the previous commit exactly. Any uncommitted work is permanently gets removed.
  - Use this, if you had made a mistake, everything is broken, and you want a completely fresh start from a specific point.
- What is *HEAD* stands for? write your observations _____

4.4.4: Addressing new requirements with help of branching, switching branch, merging changes.

After adding an index page, now you want to enhance your website by adding a new feature. You

want to add a new service page. Generally adding a new feature need lots of changes and it takes time. Hence, we create a feature branch from main branch. Once changes are complete, you will merge the changes with main branch.

- Type to check the current branch you are in: *git branch*
  What did you observe in the output? _____
- Type to rename current Branch: *git branch -M main*
- Type to create and checkout to new branch: *git checkout -b feature/add-service-page*
  - Type if you want to switch to another already existing branch: *git checkout feature/add-service-page*
- Create a new service.html and add following contents

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>service</title>
</head>
<body>
  <h1>this is service page</h1>
</body>
</html>
```

- Add, commit with proper comments
- Type to checkout to main branch: *git checkout main*
  What did you observe in the project folder in VSCode? _____
- Type to merge feature branch changes into main branch: *git merge feature/add-service-page*
  What did you observe in the output? _____
- Type to print list of commits: *git log –oneline*
  What did you observe in the output? _____

4.4.5: Concept of Stashing

You are still working on service page in feature branch. Your Team Lead walks in and ask you do changes index.html in main branch. If you try to checkout (or switch) right now, Git will block you or carry your messy, uncommitted changes over to main. How do we handle this situation? Let's see how Git helps here. It's by using technique called **Stashing**.

- Checkout to feature/add-service-page branch
- Make some new changes, let's say we will add new **app.js** file and add following contents
  alert("Welcome! The script is running.");
- Update service.html , add following contents line before </body>
  <script src=”app.js”></script>
- Type to add the changes*: git add service.html app.js*
- Observe the status using command: *git status*
- Type to stash the changes *git stash -m "WIP: started working on service page"*
  What did you observe in the project folder in VSCode? _____
  - Type if you want to drop the stash: *git stash drop*
- Switch to main branch, update index.html
  <h1>welcome to my website , page updated !!!!!!!!!!!!!!</h1>

- Add, Commit changes with proper comments
- Switch back to feature/app-service-page branch
- Type to restore those stashed changes back into branch: *git stash pop*
  What did you observe in the project folder in VSCode? _____
- Type to display list of stashes: *git stash list*
  What did you observe in the output? _____
- Add, commit changes with proper comments.
- Once changes are complete in feature/add-service-page, and tested successfully, merge with main branch.
- Observe the status using command: *git log --oneline*
  Note down the first/latest commit id and message _____

4.4.6: Handling merge conflicts

There arises situation 2/two developers made changes on same file, same line of code. Git will raise conflicts when you try to merge them and alert you, so you need to handle them manually. Here we will simulate this scenario.

- While you are in main branch, update service.html page
  <h1>this is service page, updated by main branch</h1>
  Add, commit with proper comments
- Checkout feature/add-service-page
  <h1>this is service page, updated by feature/add-service-page branch</h1>
  Add, commit with proper comments
- Checkout main page
- Merge from feature/add-service-page
  What did you observe in the project folder in VSCode? _____
- Click Resolve in Merge Editor or manually edit the product.html accordingly, choose default comment provided by Git or enter proper comments, and save
- Check commit log history: *git log –oneline*

4.4.7: Rebasing the branch

Git Rebase is an alternative way to integrate changes from one branch into another.

While Git Merge combines branches by creating a special "merge commit," Git Rebase rewrites project history by moving entire branch so it begins at the tip of another branch. It is like unplugging the Feature branch, updating the history beneath it, and plugging it back in at the very front of the timeline.

In case of Merge, objective is to bring in my feature branch changes to main branch so that I can proceed with deployment on Production. I will do this by switching into the main branch, merge feature changes into it.

In case of Rebase, objective is, while I am still working on my feature branch, some one has updated main branch and I wish bring those into my feature branch so that I can continue working on my branch. I will do this by switching into feature branch and rebase from main. This will move all my feature changes (may be into some memory temporarily), bring in changes from main, and then places feature changes (bring back from memory) on the top of it.

- Checkout main branch

- Create and checkout new feature/add-contactus-page
  Create new contactus.html and add following contents

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contactus</title>
</head>
<body>
  <h1>Contact us page</h1>
</body>
</html>
```

  Add, commit with proper comments
- Checkout main branch
- Purposely modify index.html 3 times with 3 commits. This is just to ensure multiple commits which we will be rebasing into feature branch
  - Update index.html add * section1 *
    Add, commit with proper comments. You can use single command and commit using *git commit -am "added section1"*
  - Update index.html add * section2 *
    Add, commit with proper comments *git commit -am "added section2"*
  - Update index.html add * section3 *
    Add, commit with proper comments *git commit -am "added section3"*
- Checkout branch feature/add-contactus-page
- Check commit log history: *git log –oneline*
  What did you observe in the output (top 2 lines)? _____
- Checkout feature/add-contactus-page branch
- Type to rebase : *git rebase main*
- Check commit log history: *git log –oneline*
  What did you observe in the output(top 5 lines)? _____
- Once changes are complete in feature/add-contact-page, and tested successfully
  - Checkout main branch
  - merge main branch with feature/add-contact-page changes

4.4.8: Create Personal Access Token (PAT) on github.com [one-time task]

Go to github.com > Login > Settings > Scroll down, Click Developer settings > Personal Access Token > Fine grained tokens > Generate new Token > Enter following details

- Token name
- Description
- Expiration: 90 days
- Repository access > Choose All repositories
- Permissions > Add permissions > Contents > set Contents > Choose Access: Read and write
- Click ‘"Generate Token”

System will generate new token, copy and paste and save it in a safe place

Write down your understanding of this exercise which you executed

_____

What is SSH Key?

_____

Difference between PAT and SSH Key?

_____

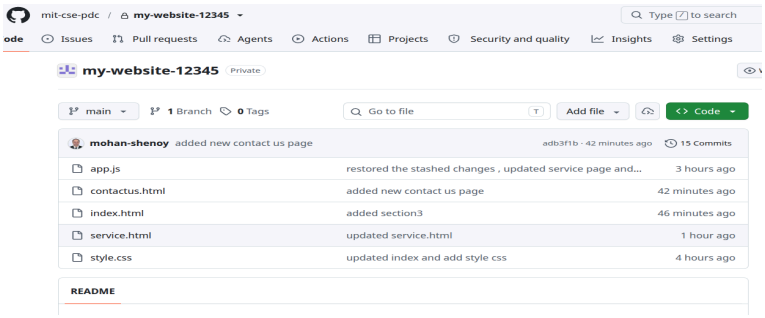
4.4.9: Pushing the project source code to github.com

Its now time to push your changes to github, a central shared server, so that it can be saved for future purposes and also allow you to work from another machine by pulling from there.

- Login to github.com
- Create new repository name as my-website-*studentregno*
  Note: Preferably same name you had given in step #4.4.1
  Description: keep it same as name
  Choose visibility: private
  Keep rest of settings: as is
  Click Create Repository
  In the confirmation page you see section "…or push an existing repository from the command line", copy 3 lines below this.
- Checkout main branch in local terminal
- Type to check if any remote repo link: *git remote –v*
- Type to set remote server: paste 1st line: line starts like this...*git remote add origin https://github.com/*.....
- Type to check if entry is added : *git remote –v*
  What did you observe in the output? _____
- Type to push the changes: paste 3rd line: *git push -u origin main*
- If all ok, you will see response something like below

```
mohan@MSP-ThinkPad-E14-Gen-5:~/CCD/my-website-12345$ git push -u origin main
Username for 'https://github.com': mohan.shenoy@manipal.edu
Password for 'https://mohan.shenoy%40manipal.edu@github.com':
Enumerating objects: 41, done.
Counting objects: 100% (41/41), done.
Delta compression using up to 12 threads
Compressing objects: 100% (38/38), done.
Writing objects: 100% (41/41), 4.11 KiB | 1.03 MiB/s, done.
Total 41 (delta 20), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (20/20), done.
To https://github.com/mit-cse-pdc/my-website-12345.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
mohan@MSP-ThinkPad-E14-Gen-5:~/CCD/my-website-12345$
```

- Go back to new repository you had created earlier, check if all latest changes have been pushed



4.4.10: Clone the project source code from github.com

You can now clone the repo from some other machine and make new changes.
Just to show how it works, let us clone the repo on the same machine.

- In the repository page, click "<> Code" button (green color) and copy Clone > Https > URL
- Create new directory on local machine
- Open terminal and type to clone the Repo: *git clone <<paste copied URL here>>*
- Enter email address
- Enter PAT Token
- Source code will be downloaded
- What does git clone command do? _____

4.4.11: Cherry Picking (Additional)

Read Git documentation, use this feature and document your understanding

4.4.12: Squashing Commits (Additional)

Read Git documentation, use this feature and document your understanding

4.4.13: Git Tags (Additional)

Read Git documentation, use this feature and document your understanding

4.4.14: Blame and Log Filtering (Additional)

Read Git documentation, use this feature and document your understanding

4.5 References:

- Git commands https://git-scm.com/book/en/v2
- Pro Git ebook : https://github.com/progit/progit2/releases/download/2.1.450/progit.pdf

4.6 Remarks:

- Ensure you have pushed all changes to GitHub
- Delete local project folders
- Remember to save your PAT (Token) (created in Exercise #4.4.8) so that it can used in next lab

LAB #5

LAB #5: WORK IN PROGRESS

5.1 Objectives:

5.2 Theory/Concepts:

5.3 Solved Examples:

5.4 Exercises:

5.5 References:

5.6 Remarks:

REFERENCES:

1	